

# 实验四：软件分析

（项目内容：开发一款简洁高效的任务管理应用，帮助南京大学校内用户管理日常任务，提供任务分类与标签设计、任务时间线一览、惩罚与激励机制、任务的截止日期提醒等功能）

## 1. 实验目的

- 掌握使用分析工具检测项目缺陷的方法
- 评估工具在正确报告和误报方面的表现
- 加深对程序分析技术的理解，提高代码质量和安全性

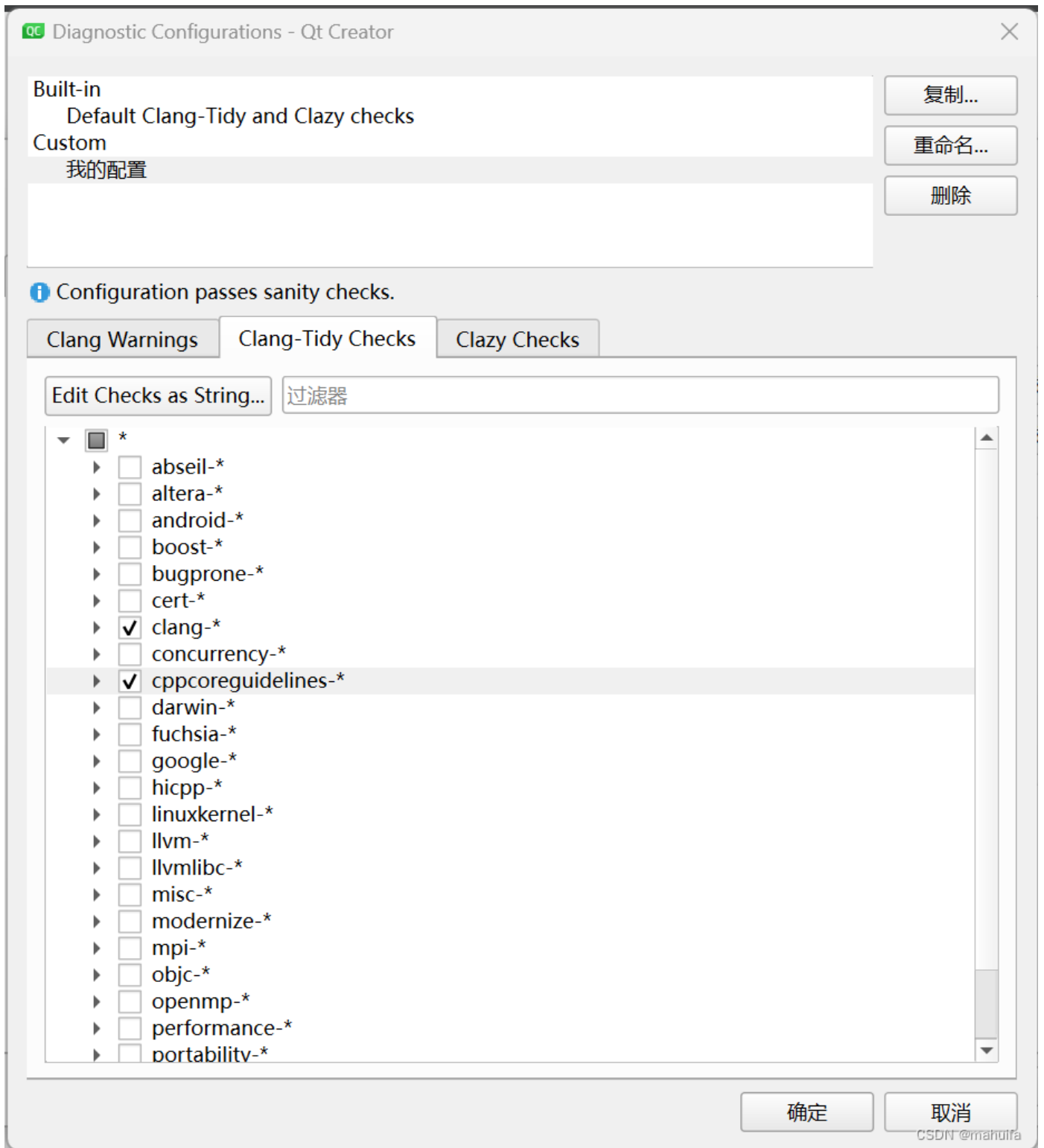
## 2. 实验工具和方法

### 2.1 工具选择

- 静态分析工具：Clang Static Analyzer、Cppcheck
- 大语言模型：ChatGPT

### 2.2 配置与运行方式

- 静态分析工具：
  - Qt中有内置的静态分析工具 Clang-Tidy，但是需要自行配置。Default Clang-Tidy and Clazy checks 配置是内置配置，无法进行修改，点击【复制】重新创建一份配置就可以修改了，选择【Clang-tidy Checks】，勾选自己设置的检查项（在本次实验中我选择了 clang 和 cppcoreguidelines，分别代表Clang 编译器的诊断规则和C++ 标准委员会制定的一组建议）



- 大语言模型：

- 下面是给 ChatGPT 提供的一个清晰且详细的 prompt 模板

以下是一个 C 语言项目的代码片段。请对代码进行全面分析，找出潜在的缺陷并提供详细的报告，包括缺陷的类型、位置、影响范围和修复建议。我们特别关注以下几种问题：

1. 内存泄漏：是否有未释放的内存？
2. 空指针引用：是否有可能在空指针上调用函数或访问数据？
3. 数组越界：是否有可能访问数组的非法索引？
4. 资源泄漏：是否有文件、套接字、内存等资源没有正确释放？
5. 代码风格：是否有违反编码规范的地方，比如变量命名、函数设计等？
6. 安全漏洞：是否有潜在的缓冲区溢出、SQL 注入、格式字符串漏洞等安全隐患？

请检查每一个问题，并输出以下格式的详细报告：

1. **\*\*缺陷类型\*\***（例如：空指针引用、内存泄漏等）
2. **\*\*影响范围\*\***（例如：可能导致程序崩溃、可能导致未定义行为等）
3. **\*\*文件\*\***：问题所在的文件路径（例如：`src/main.c`）
4. **\*\*行号\*\***：问题出现的代码行号
5. **\*\*问题描述\*\***（例如：代码在这里尝试对空指针进行解引用）
6. **\*\*修复建议\*\***（例如：在调用前检查指针是否为空）

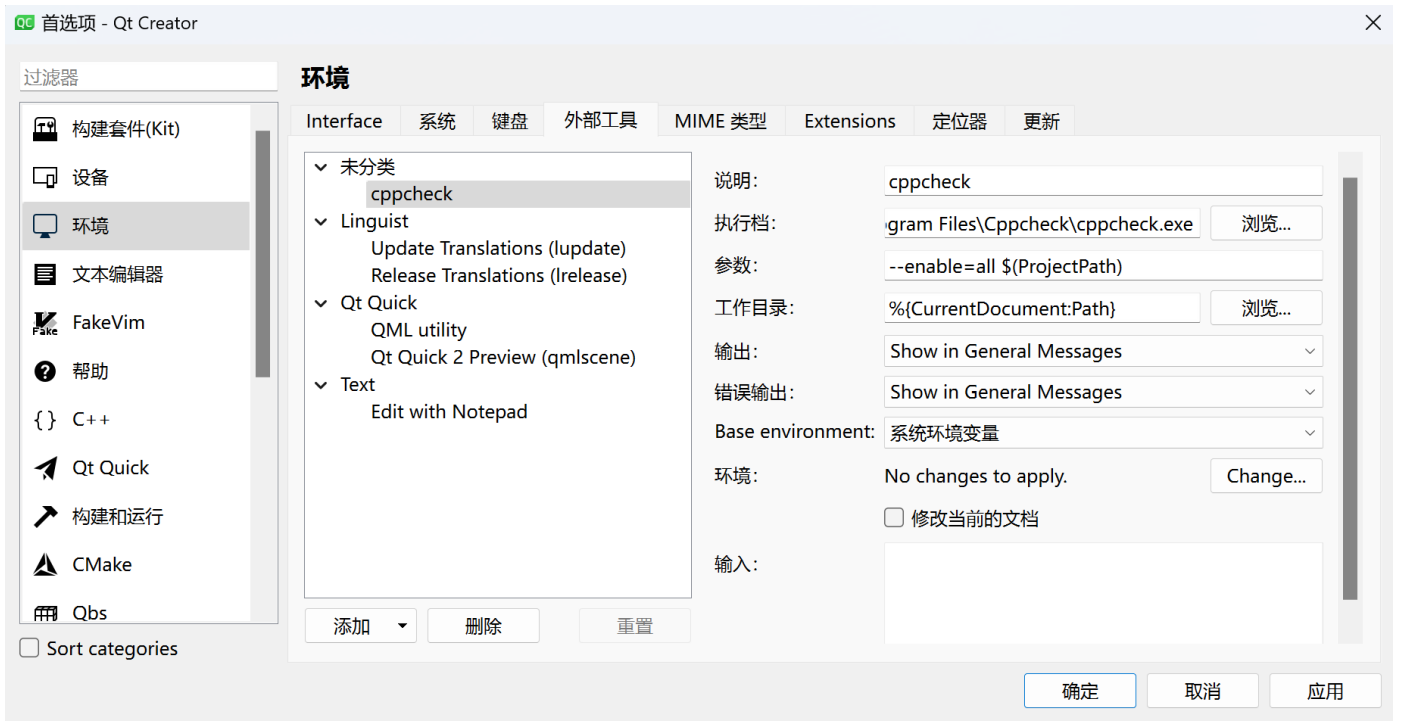
**\*\*代码片段\*\***：

<在这里插入项目代码>

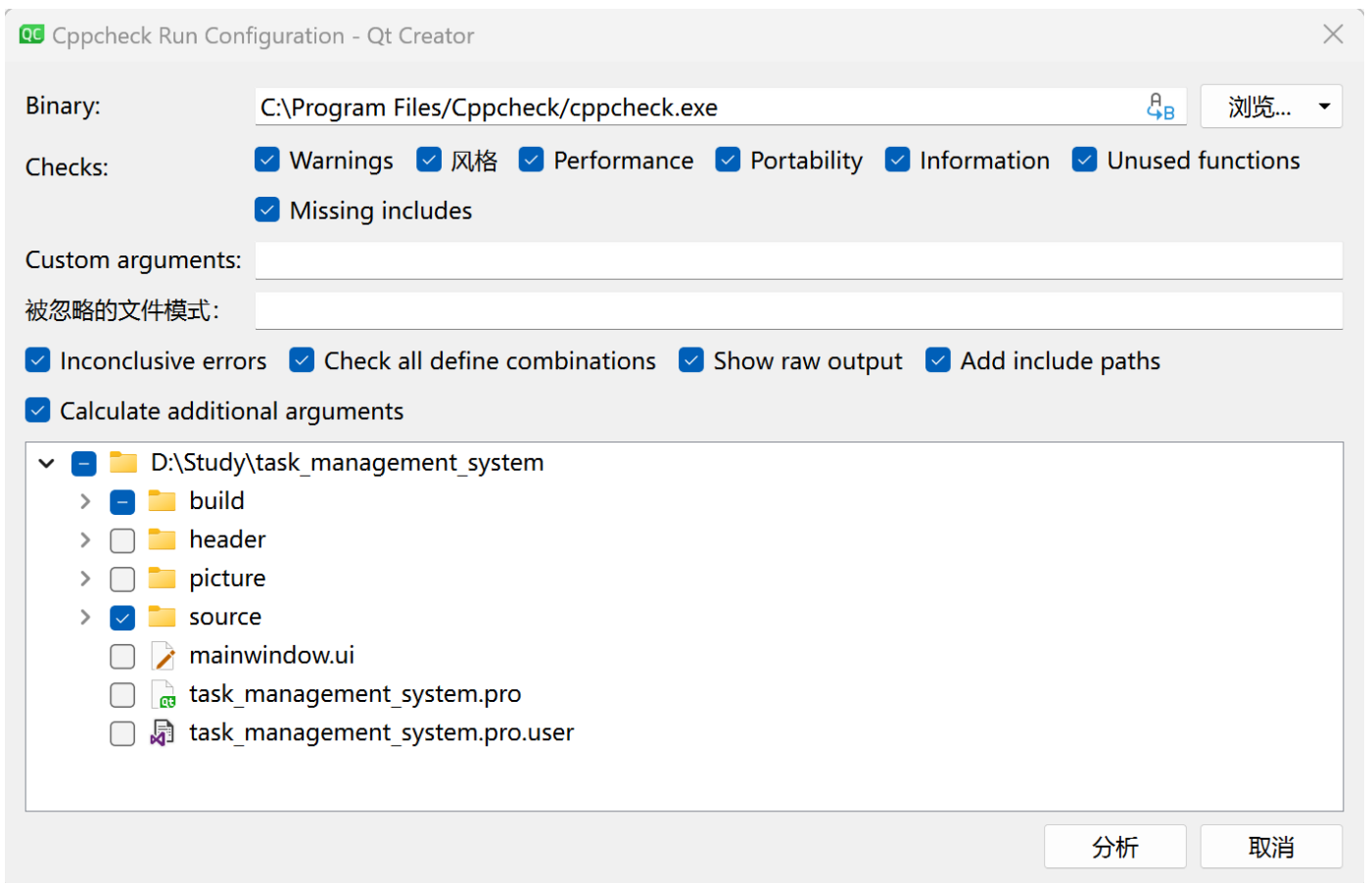
如果有多个问题，逐个列出，最好按严重性进行排序。请确保报告包括每个问题的详细信息，并提出具体的修复措施。如果在代码中发现了误报（**false positive**），请指出并说明原因。

- **形式化工具（替换为了另一个静态分析工具）：**

- 因为 Qt 项目通常包含大量的框架特定代码（例如，信号和槽机制、元对象系统、UI 模块等），这些代码在标准 C++ 工具中无法直接解析或验证，需要删除与 Qt 框架强绑定的代码，为无法移除的部分编写模拟代码等等，工作量太大（例如用普通 C++ 函数替代信号与槽机制），所有在此另选一个静态分析工具 `Cppcheck`
- 由于 `Cppcheck` 属于外部工具，所以需要自行安装后，在 Qt 中手动导入



- 如图选择检查项，由于 `cppcheck` 在实际表现中稍差，本次实验中勾选了全部检查项（比如性能、警告、可移植性等）



## 3. 实验过程

### 3.1 植入代码缺陷

- 由于实验三写代码时候写的很认真，而且有ChatGPT和UML图的辅助，所以在非UI逻辑这边没有任何明显的缺陷（代码量在1000行左右），所以不得已在三个规模最大的文件中植入缺陷(共八个)，接下来的缺陷分析也将集中在 `ScheduleManager.cpp`、`Task.cpp` 和 `User.cpp` 这三个文件当中(包括代码本来就存在的一些规范警告)
- 实验中医入的代码缺陷类型（七类）
  - CWE-401 (Memory Leak): 内存泄漏（2）

```

21  User::~~User() {
22      saveTasks("tasks.json");
23      saveTags("tags.json");
24
25      //忘记释放 `reminderTimer`，导致内存泄漏
26      //delete reminderTimer;
27  }

```

```

32  bool Task::createReminder(const QDateTime& reminderTime, const QString& notificationMethod) {
33      Reminder* reminder = new Reminder(); // 动态分配内存 Δ initializing non-owner 'Reminder'
34      reminder->setReminder(reminderTime, notificationMethod, title);
35      reminders.append(*reminder); // 将其添加到 QList 中，但并没有删除这个指针
36      // 忘记释放 `reminder`，导致内存泄漏
37      return true;
38  }

```

- CWE-415 (Double Free): 双重释放（1）

```

70  bool Task::deleteReminder(Reminder* reminder) { ⓘ Parameter
71      for (int i = 0; i < reminders.size(); ++i) {
72          if (reminders[i] == *reminder) {
73              // 错误：删除指针两次
74              delete &reminders[i]; // 删除对象，但它是栈上的对象
75              reminders.removeAt(i); // 再次移除对象
76              return true;
77          }
78      }
79      return false;
80  }

```

- CWE-476 (NULL Pointer Dereference): 空指针解引用（1）

```

8   bool Task::editTask(const QString& newTitle, const Tag&
9   |                   |                   |                   |
10  |                   |                   |                   | const QDateTime& newDeadline, int r
11  |                   |                   |                   | if (newTitle.isEmpty() || !newDeadline.isValid()) {
12  |                   |                   |                   |     return false;
13  |                   |                   |                   | }
14  |                   |                   |                   | // 假设 tag 是一个可能为空的指针
15  |                   |                   |                   | Tag* pTag = nullptr; // 空指针
16  |                   |                   |                   | pTag->setName(newTag.getTagInfo()); // 空指针解引用
17  |                   |                   |                   | title = newTitle;
18  |                   |                   |                   | tag = newTag;
19  |                   |                   |                   | description = newDescription;
20  |                   |                   |                   | deadline = newDeadline;
21  |                   |                   |                   | priority = newPriority;
22  |                   |                   |                   | this->status = status;
23  |                   |                   |                   | return true;
24  |                   |                   |                   | }
25

```

- CWE-787 (Out-of-bounds Write): 写入越界内存 (1)

```

16  bool SchedulerManager::createTag(const QString& name) {
17  |   if (!name.isEmpty() && name.length() <= 4) {
18  |       bool flag = true;
19  |       // 防止用户重复建立相同标签
20  |       for (Tag tag : getTags()) { ΔMissing reference in range-for wi
21  |           if (tag.getTagInfo() == name) flag = false;
22  |       }
23  |       if (flag) {
24  |           Tag newTag;
25  |           newTag.setName(name);
26  |           tags.append(newTag);
27  |
28  |           // 引入越界错误: 访问超出范围的索引
29  |           int invalidIndex = tags.size() + 100; // 故意创建一个不合理的索引
30  |           if (invalidIndex >= 0 && invalidIndex < tags.size()) { i C
31  |               tags[invalidIndex] = newTag; // 这是一个越界的写入
32  |           }
33  |
34  |           return true;
35  |       }
36  |   }
37  |   return false;
38  }

```

- CWE-416 (Use After Free): 使用已经释放的内存 (1)

```

62 bool SchedulerManager::deleteTag(const QString& name) {
63     Tag* tagToDelete = nullptr;
64     for (int i = 0; i < tags.size(); ++i) {
65         if (tags[i].getTagInfo() == name) {
66             tagToDelete = &tags[i];
67             break;
68         }
69     }
70
71     // 引入 use after free 错误
72     if (tagToDelete) {
73         tags.removeOne(*tagToDelete);
74         delete tagToDelete; // 删除指针（不必要的操作）
75     }
76
77     // 错误地访问已删除的内存
78     if (tagToDelete != nullptr) {
79         qDebug() << tagToDelete->getTagInfo(); // 使用已删除的;
80     }
81
82     return false;
83 }

```

- CWE-665 (Improper Initialization)：未正确初始化变量（1）

```

120 // 引入 Improper Initialization 错误
121 Tag uninitializedTag; // 未初始化的标签
122 for (const QJsonValue& value : tagArray) {
123     if (value.isObject()) {
124         QJsonObject tagObject = value.toObject();
125         QString name = tagObject["name"].toString();
126         createTag(name); // 使用 createTag 方法来添加标签
127
128         // 错误使用 uninitializedTag, 它未被正确初始化
129         qDebug() << uninitializedTag.getTagInfo(); // 这里使用了一个未初始化的标签
130     }
131 }

```

- CWE-190 (Integer Overflow)：整数溢出(1)

```

40 | bool User::createTask(const QString& title, const QString& tagName, // The function
41 |                     const QString& description, const QDateTime& deadline, int priority,
42 |                     bool reminder_flag, const QDateTime& reminderTime) {
43 |     createTag(tagName);
44 |     QList<QString> tagNames;
45 |     for (Tag tag : getTags()) { ⚠️ Δ Missing reference in range-for with non trivial type (Tag)
46 |         tagNames.append(tag.getTagInfo());
47 |     }
48 |
49 |     // 故意引入整数下溢 (CWE-191)
50 |     int invalidPriority = priority - 100000; // 这里我们故意让优先级下溢，导致非常大的正值
51 |
52 |     // 先创建标签，如果标签没有创建成功（标签列表里面没有该标签），那就算创建任务失败
53 |     Task newTask;
54 |     if (tagNames.contains(tagName)) {
55 |         if (newTask.editTask(title, getTags()[tagNames.indexOf(tagName)], description, deadline,
56 |                             if (reminder_flag) newTask.createReminder(reminderTime, "");
57 |                             tasks.append(newTask);
58 |                             return true;
59 |     }

```

## 3.2 静态分析工具（Clang-Tidy）的检测结果

- 真实存在缺陷：空指针解引用、双重释放、使用已经释放的内存
- 误报缺陷（没有对程序造成影响的**规范警告**）：魔法数字、初始化顺序、**不正确的内存管理**
- 其实误报中的**不正确内存管理**，就是我植入的两个内存泄露的错误，所以也可以算是检测成功。`reminder` 是一个非所有者指针，但是它指向了一个通过 `gsl::owner<>` 初始化的新对象。也就是说，`reminder` 是一个不负责管理内存的指针，但是它指向的是一个由 `gsl::owner<>` 负责管理内存的对象。这样就会导致内存管理不当，因为 `reminder` 不会释放该对象的内存，而 `gsl::owner<>` 则可能会释放它，造成 `double free` 或内存泄漏的问题
- 至于**魔法数字**和**初始化顺序**，在这里也解释一下。初始化顺序说的是，`reminderTimer` 应该在构造函数的成员初始化列表中进行初始化，而不是在构造函数的函数体内进行初始化。魔法数字说的就是代码中直接写入的硬编码常量，缺乏解释或上下文，通常被认为是不好的编程习惯，比较难维护

## 3.3 大语言模型（ChatGPT）的检测结果

- 真实存在缺陷：空指针引用、内存泄漏、双重释放、**缺乏输入验证**、数组越界、使用已删除的内存、未正确初始化变量
- 误报缺陷（没有对程序造成影响的规范警告）：代码风格问题、资源泄漏、数组越界、安全漏洞、逻辑错误
- 难以相信啊，`ChatGPT` 除了找到我植入的所有错误以外，甚至还发现了一个我在项目中没有专门去干的事——**输入验证**，其实这个才是造成我整数溢出类漏洞的根本原因。不过，它发现的错误有点过于宽广了，而且正确性也有待考证，就比如下面这个规范警告，其实是错误的（而且实验三中已经通过了Google编程c++的风格统一）；还有很多极其荒诞的错误，就比如我初始化了一个标签，调用的是默认构造函数，但ChatGpt却认为我被初始化为了一个 `nullptr`，进而引发了后面的一串分析，这些会在结果与分析中会提到原因



## 6. 缺陷类型：代码风格问题

- **影响范围**：影响代码可读性和维护性
- **文件**：Task.cpp 等
- **行号**：不特定（代码整体）
- **问题描述**：部分命名没有遵循一致的编码风格。比如，reminderTimer 和 reminder 使用了不同的命名方式。根据项目的编码规范，应统一命名风格，使用驼峰命名法或其他一致的命名规范。
- **修复建议**：统一命名风格，如所有成员变量使用 camelCase 命名。

## 3.4 静态分析工具（Cppcheck）的检测结果

- 真实存在缺陷：空指针解引用、双重释放
- 误报缺陷（没有对程序造成影响的规范警告）：缺少头文件、未使用的函数、不清楚的标识符、缺少构造函数
- 在代码中，Task 类有一些私有成员变量，Cppcheck 检测到这些成员变量需要初始化，但是 Task 类没有定义构造函数。能够检测到这个缺少构造函数缺陷让我感到有些惊讶，因为根据 Cppcheck 的整体表现，它是远不如前两个好的，但是它却检测出了我代码为数不多的逻辑错误（具体实现是用其它的方式避开了构造函数的需要，但后面思索这样子的实现其实是不太合适的），感觉大概是因为这样的实现方式不符合C++ 标准委员会制定的建议，所以就被检测出来了（事实也证明了这样子实现的不方便）

## 4. 结果分析与评估

### 4.1 误报率评估

- 就从我植入的八个缺陷来看（虽然比较少，但也能在一定程度上反映），ChatGPT 高居榜首，无论是正确率还是误报率，ChatGPT 都高居第一位，属于是风浪越大鱼越大的一种，需要仔细甄别
- Clang-Tidy 检测出的真实缺陷是比 Cppcheck 多的，误报率也要低于 Cppcheck，但是偏偏 Cppcheck 在潜在的错误检测方面就是比 Clang-Tidy 好，会在下一部分尝试分析原因

### 4.2 误报原因分析

- 从代码复杂度的角度来看，这个分析的代码规模比较大，非UI逻辑分布在了十几个文件中，代码量总共有1000行左右，而且其中还涉及到了Qt的框架，所以像 Cppcheck 这种没有一开始就集成到Qt中的工具，以及像 ChatGPT 这种需要详细上下文的工具，在没有铺垫好的情况下，表现情况其实是不尽如人意的。就比如 Task 类利用一些其它的方式避开了构造函数的需要，这种是跨文件的（分析文件和用到的函数不在一个文件中），就极其容易给静态分析工具造成误导

- `Cppcheck` 是一个静态代码分析工具，它主要专注于发现潜在的编程错误、代码风格问题。然而，关于内存管理的错误检测，`Cppcheck` 并不是专门针对内存泄漏、越界访问、空指针等问题进行深度分析的工具。它的设计理念更侧重于检测逻辑错误（诸如潜在的空指针解引用、无效的循环、未使用的变量等）、代码风格规范、很简单的内存管理等，所以在内存方面的缺陷检测不尽如人意。但是，它能发现Clang-Tidy和大语言模型都没发现的逻辑“错误”，让人不得不感慨C++委员会制定的建议的合理性
- `ChatGPT` 的话，它的表现，很大程度上是受到上下文信息以及你的使用习惯影响的，所以如果代码片段没有给足或者要求不清晰，就很容易出错。就比如这次实验，只给了它三个有缺陷的代码文件，那剩下的代码它就只能根据命名去自己猜测，很容易忽略一些具体的实现，从而造成误报。还有就是ChatGPT为了可维护性和全面性，很可能会提出一些预防建议，而这些在你的代码中是不会对程序运行造成影响的，一定程度上也会造成误报。

## 5. 结论

### 5.1 各工具检测能力对比

| 工具/模型   | Clang-Tidy                      | Cppcheck                                           | 大语言模型                |
|---------|---------------------------------|----------------------------------------------------|----------------------|
| 检测类型    | 语法、代码规范、潜在的运行时错误、内存管理、性能、C++特性  | 简单的内存错误、空指针解引用、代码风格、变量初始化等                         | 静态分析错误的缺陷、代码         |
| 主要优点    | 集成到编译流程、强大的 C++ 风格检查、对内存管理有较强支持 | 独立于编译器、广泛的静态代码分析，支持多种编程语言                          | 生成式，能够误解释和建议         |
| 缺点      | 需要集成到编译过程中，可能对复杂代码不够全面          | 设计理念简单、检查范围有限、对复杂代码结构的分析能力不足                       | 主要提供建议码，依赖于上         |
| 检测速度    | Clang-Tidy的速度通常较快               | Cppcheck是一个独立的工具，分析过程并不依赖于编译器的编译过程，通常比Clang-Tidy更快 | 从总耗时来看度确实远不如竟还需要上下啥的 |
| 检测误报率   | 误报率较低，但对一些细节可能会漏掉               | 误报率较低，但可能对一些复杂模式分析不足                               | 误报率高，取上下文的清晰         |
| 代码风格和规范 | 对代码风格、C++最佳实践的强力支持              | 支持简单的代码风格检测，较少涉及复杂的风格规则                            | 基于自然语言问题提供代码         |
| 性能优化    | 提供性能建议，如无用代码、数据结构选择等            | 较少关注性能问题                                           | 能够提供高层议，但缺乏组         |

### 5.2 工具使用建议

- **Clang-Tidy**是一个适用于C++项目的静态分析工具，尤其适合那些对代码质量有严格要求的项目。最重要的是！它可以**和Qt工具集成**！也就是说可以检测到Qt框架，从而自动化检测代码风格和潜在错误（比如内存泄漏、空指针解引用等），并提供具体的改进建议。特别是在**内存管理和性能优化**方面，**Clang-Tidy**显得特别有效。建议将其与其他工具结合使用，可以弥补其在逻辑错误的盲区，尤其是与**Cppcheck**搭配时，能够更全面地覆盖不同类型的错误
- **Cppcheck**是一款跨平台的静态分析工具，专注于发现潜在的错误。然而，它在内存管理方面的检测能力并不如一些专门的工具（就比如**Clang-Tidy**）。Cppcheck更适合用于发现不容易察觉的潜在问题，例如**简单的逻辑和内存错误**。建议将其与**Clang-Tidy**和**ChatGPT**配合使用，特别是在内存相关的问题上，增强其检测能力。除此以外，大语言模型还可以补充其在复杂逻辑错误和代码重构方面的不足
- 至于**ChatGPT**，说实话这一年我也借用它写了许多代码了，我到现在都没有试探出它的上限在哪，我感觉只要给足它**上下文**和**自己在意的点**，在代码处理这部分，表现的很多时候是要比自己强的。唯一给我造成困扰的就是，如何能把信息准确传达给它（就比如语言表达稍显情绪化或者是代码规模比较大），但就单论这次实验，在**仔细甄别**回答的前提下，表现是比另外两个静态分析工具要好的，而且上手难度较低。建议在开发后期使用，与**Cppcheck**配合使用，能够在基础静态分析的基础上提供更为深入的逻辑分析和设计优化建议，提升整体缺陷检测的能力

## 5.3 工具组合推荐

- 单单从本次实验来看，个人还是比较推荐**Cppcheck + ChatGPT**的组合。一方面是这个组合的合理性，**Cppcheck**提供基础的静态分析，简单的逻辑内存错误等，ChatGPT进一步深入代码的语义层面，提供更详细的逻辑缺陷分析，内存相关方面的问题和代码重构方面的不足。另一方面就是**ChatGPT**在上下文铺垫好以后，虽然误报率高，但是它成功率也高啊，对于内存管理的能力着实是比**Clang-Tidy**好不少，而且使用起来两者的难易程度更是不在一个水平，所以果断就pass掉**Clang-Tidy**了

## 6. 对这个实验一些想说的话

- 说实话，我真的觉得这个实验设计的远远不如实验二，就给人一种为了完成实验而来设计这个实验的感觉，如果是想让同学们了解软件分析工具的使用，了解一些静态分析的漏洞，完全可以官方给出一份有着大小bug的代码，然后让同学们都对着这一份代码去分析呀，这样子不仅有成就感，而且也可以根据同学们发现错误的数量和分析错误的全面程度来给分。像这种当初写代码写的很仔细，最后没有缺陷还要在代码里面自行植入缺陷的操作，还要根据这些植入的数量很少的缺陷评估误报率，除了能让大家熟悉软件分析工具以外，搞不懂有何意义所在